



Manual de Implementação: Visão do Cozmo

A implementação da câmara no ZoyBlocks divide-se em **4 Fases Críticas**. O objetivo é transformar luz capturada pelo robô em frames visíveis na interface web do Electron.

Fase 1: O Servidor de Hardware (Python)

Objetivo: Capturar a imagem bruta e convertê-la num formato que viaje rapidamente pela rede.

- **O Conceito de Callback:** Usamos o `cli.add_handler`. Em vez de o programa "perguntar" se há imagem, o robô "avisa" quando um frame novo chega.
 - **Processamento de Imagem: * Compressão JPEG:** Imagens brutas são pesadas. Reduzimos a qualidade para 70% usando a biblioteca `Pillow` para garantir fluidez.
 - **Base64:** Transformamos os bytes da imagem em texto. Isso permite enviar a imagem dentro de um JSON pelo terminal (`stdout`), sem precisar de criar ficheiros temporários.
-

Fase 2: O Túnel de Dados (Electron Main & Preload)

Objetivo: Atuar como o "carteiro" que leva a informação do processo Python para a interface.

- **Main Process (`main.js`):** * O Electron vigia o `stdout` do Python. Quando vê uma mensagem do tipo `camera`, ele dispara um evento `mainWindow.webContents.send`.
 - **Segurança:** O Electron garante que o processo Python só é acedido através de canais controlados.
 - **Preload Script (`preload.js`):**
 - Cria a ponte `window.electronAPI.onCozmoLog`. Sem isto, o JavaScript da página não teria permissão para "ouvir" o que o robô está a dizer.
-

Fase 3: A Interface do Utilizador (Renderer)

Objetivo: Exibir a imagem e gerir o estado do hardware através de botões.

- **Renderização Dinâmica:** No `home.js`, pegamos na string Base64 e injetamos diretamente no `src` da tag `<img>`. O navegador interpreta o texto como uma imagem instantaneamente.
 - **Hierarquia de Estados (UX):**
 - O botão da câmara nasce **desativado** (`disabled`).
 - Só é libertado após o log `{"status": "ready"}`.
 - **Porquê?** Para evitar que o sistema tente "ver" antes de o robô estar ligado, o que causaria erros de memória.
-

Fase 4: O Ciclo de Vida e Performance

Objetivo: Manter o sistema rápido e estável.

- **Ativação sob Demanda:** A câmara só consome CPU e bateria quando o utilizador clica em "Ligar". Quando desligada, o Python remove o handler (`remove_handler`), parando o processamento.
- **Controle de Lag:** Ao usar `PYTHONUNBUFFERED=1` no `main.js`, garantimos que os frames não fiquem presos num "túnel" de memória, chegando em tempo real à tela.



Resumo Estrutural para o Aluno

Etapa	Componente	Tecnologia	Função Principal
1. Captura	cozmo_server.py	Python / PyCozmo	Transforma frames em Base64.
2. Envio	stdout	System Pipe	Transporta o JSON com a imagem.
3. Receção	main.js	Node.js / IPC	Escuta o terminal e repassa ao Front-end.
4. Exibição	home.js	JavaScript / HTML	Atualiza a tag <code></code> na tela.